

can also send values to the kernel sub-system and can control the internal settings. This is how programs talk to the Linux kernel.

Sysfs mounting

By default, *sysfs* is compiled in the Linux kernel. It is dependent on *CONFIG_SYSFS* being enabled in the kernel configuration. If *sysfs* is not already mounted, then you can do so by using the following command:

```
mount -t sysfs sysfs /sys
```

Linux Kernel objects

Folders in each *sysfs* directory are represented by kernel objects (kobjects) in Linux. These are represented in the kernel through the *struct kobject* (defined in *include/linux/kobject.h*). The important members of the *struct kobject* are given below.

char *name: This is the name of the kobject. Folders corresponding to the current kobject are created with this name in *sysfs*.

Struct kref *parent: This is the parent of the current kobject being created. When a folder is created in *sysfs*, if this field is present, then the current kobject folder is created inside the parent kobject folder.

Struct kref: This is the reference counting mechanism for kobject. Whenever any kernel module refers to any kobject, its reference count is incremented, and whenever any kobject reference is released by any kernel module, then the reference count is decremented. When the reference count decrements to zero, memory related to the kobject is released.

Directory operations in sysfs

The following are the interfaces that are used for directory related operations in *sysfs*:

```
int sysfs_create_dir(struct kobject *kobj);
```

```
void sysfs_remove_dir(struct kobject *kobj);
```

```
int sysfs_rename_dir(struct kobject *kobj, const char *new_name);
```

Sysfs directory creation

The *sysfs_create_dir* interface is used to create a directory in *sysfs*. The *struct kobject* parameter passed to it has two fields in it.

kobj->name: This is the name of the directory that is to be created in *sysfs*.

kobj->parent: This is the kobject pointer of the parent directory in which the current directory is to be created.

Since each directory in *sysfs* has a *struct kobject* associated with it, when the new directory is to be created in

sysfs, the *struct kobject* pointer that is passed to *sysfs_create_dir* should have the *parent kobject* pointer pointing to the kobject of the directory in which the current directory is to be created. Then the name parameter of the kobject should be filled, with the proper name representing the functionality for which the directory is being created.

But the above interfaces are not directly used to create directories in *sysfs* when writing kernel modules; instead, the following interface is used:

```
struct kobject * kobject_create_and_add(const char *name,
struct kobject *parent);
```

...where,
name: is the name of the directory to be created in *sysfs*, and...

parent: is the kobject of the parent directory in which the current directory is to be created.

The return value of the *kobject_create_and_add* is the kobject pointer of the created directory, and if it fails to create a directory then a NULL pointer is returned.

Kobject parent pointer: Since each folder in the *sysfs* file system is represented using a *struct kobject*, to create a new directory in *sysfs*, the parent pointer is to be initialised with the kobject of the directory in which the current directory is to be created. For example, to create a directory named *example_dev* in */sys/kernel*, the following function call is used:

```
struct kobject *example_dev_kob;
```

```
example_dev_kob = kobject_create_and_add("example_dev",
kernel_kobj);
```

...where, *kernel_kobj* is the kobject pointer of the kernel directory in *sysfs*. So in order to create a directory in */sys/firmware*, the *firmware_kobj* pointer will be used.

In order to create the directory named *example_dev* directly in */sys*, the following function call is used:

```
kobject_create_and_add("example_dev", NULL);
```

...so if the parent pointer is kept NULL, then the *example_dev* directory will be created in the */sys* folder.

Sysfs directory removal

The *sysfs_remove_dir* interface is used to remove the directory from the *sysfs* file system. The parameter that is passed to the *sysfs_remove_dir* is the kobject pointer of the directory which is to be removed. The kobject pointer should be used to create the directory. Here, too, the same concept applies. Instead of directly calling this interface to remove the directory, the following kernel kobject interface is used, which internally calls the *sysfs_remove* interface: